INDO ASIAN ACADEMY DEGREE COLLEGE

KALYAN NAGAR, BANGALORE

(Recognized by Govt. of Karnataka, Approved by AICTE, Affiliated to Bengaluru North University)

DAA Lab (NEP Syllabus)

Part A & B

V Semester BCA

Lab Manual

<u>Prepared By</u>

M.R. Hidayathulla

Vice Principal

Indo Asian Academy Degree College

Kalyan Nagar, Bangalore - 560043

Lab 1: Write a program to sort a list of N elements using Selection sort technique.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

 int min(int a[] ,int k , int n)
{
    int loc, j , min;
    min = a[k];
    loc = k;

    for(j=k+1;j<n;j++)
    {
        if(min>a[j])
        {
            min=a[j];
            loc=j;
        }
    }

        return loc;

}

int main()
{
    int i, *arr, k, n, loc=0,temp;

    printf("Enter the number of elements :");
    scanf("%d",&n);

    arr=malloc(n*sizeof(int));

    printf("Enter the elements :\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }

    for(k=0;k<n;k++)
    {
        loc=min(arr,k,n);
        temp=arr[k];
        arr[k]=arr[loc];
```

INDO ASIAN ACADEMY

```
        arr[loc]=temp;
    }

    printf("the sorted arrays is : ");
    for(i=0;i<n;i++)
        printf(" %d ",arr[i]);


    return 0;
}
```

Output:

```
Enter the number of elements :5
Enter the elements :
10 30 20 50 40
the sorted arrays is :  10  20  30  40  50
Process returned 0 (0x0)   execution time : 14.905 s
Press any key to continue.
```

INDO ASIAN ACADEMY

Lab 2 : Write a Program to perform Travelling Salesman Problem

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>

#define MAX_CITIES 10

int n;
int distances[MAX_CITIES][MAX_CITIES];
int memo[MAX_CITIES][1 << MAX_CITIES];
int parent[MAX_CITIES][1 << MAX_CITIES];

// function to find shortest tsp using dynamic programming

int tsp(int current, int visited)
{
    if(visited == (1 << n)-1)
    {
        // All cities have been visited

        return distances[current][0];
    }

    if(memo[current][visited]!=-1)
    {
        return memo[current][visited];
    }

    int minDistance = INT_MAX;
    int nextCity = -1;
    int next;

    for(next=0;next<n;next++)
    {
        if(!(visited & (1<<next)))
        {
            int newDistance =
distances[current][next]+tsp(next,visited | (1<<next));
            if(newDistance < minDistance)
            {
                minDistance=newDistance;
                nextCity=next;
            }
        }
    }

    parent[current][visited]=nextCity;
```

INDO ASIAN ACADEMY

```
    memo[current][visited]=minDistance;
    return minDistance;
}

// function to print the TSP route
void printRoute()
{
    int current=0;
    int visited = 1;
    int i;

    printf("Optimal Route : A");

    for(i=1;i<n;i++)
    {
        int next = parent[current][visited];
        printf(" -> %c", 'A' + next);
        current=next;
        visited |=(1<<next);
    }

    printf(" -> A \n");
}

int main()
{
    printf("Enter the number of cities :");
    scanf("%d",&n);

    if(n<2 || n>MAX_CITIES)
    {
        printf("Please choose a number between 2 and %d.
\n",MAX_CITIES);
        return 1;
    }
    int i , j;
    printf("Enter the distances between cities : (cost matrix row
wise) \n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&distances[i][j]);
        }
    }
    for(i=0;i<n;i++)
    {
```

INDO ASIAN ACADEMY

```
        for(j=0;j<(1<<n);j++)
        {
            memo[i][j]=-1;
        }
    }

     int shortestDistance = tsp(0,1);
     printf("Shortest Distance : %d \n",shortestDistance);
     printRoute();

    return 0;

}
```

Output :

```
Enter the number of cities :4
Enter the distances between cities : (cost matrix row wise)
0 10 15 20
5 0 9 10
6 3 0 12
8 8 9 0
Shortest Distance : 35
Optimal Route : A -> B -> D -> C -> A

Process returned 0 (0x0)   execution time : 27.956 s
Press any key to continue.
```

INDO ASIAN ACADEMY

Lab 3 : Write a program to implement Dynamic Programming algorithm for the 0/1 Knapsack problem.

```
#include <stdio.h>

#define MAX_IEMS 100
#define MAX_CAPACITY 100

int max(int a, int b)
{
   return (a>b)?a:b;
}

int knapsack(int values[], int weights[], int n, int capacity)
{
    int dp[MAX_IEMS+1][MAX_CAPACITY+1];
    int i,j;

    //initialize the dynamic programming table

    for(i=0;i<=n;i++)
    {
        for(j=0;j<=capacity;j++)
        {
            if(i==0||j==0)
            {
                dp[i][j]=0;
            }else if(weights[i-1]<=j)
            {
                dp[i][j]=max(dp[i-1][j],dp[i-1][j-weights[i-
1]]+values[i-1]);

            }else
            {
             dp[i][j]=dp[i-1][j];
            }
        }
    }

    int result=dp[n][capacity];
    int remaining_capacity=capacity;

    printf("Selected Items:");

    for(i=n;i>0 && result>0;i--)
    {
        if(result!=dp[i-1][remaining_capacity])
        {
```

INDO ASIAN ACADEMY

```
            printf("Item %d  ",i);
            result -= values[i-1];
            remaining_capacity-=weights[i-1];
        }
    }
    printf("\n");

    return dp[n][capacity];

}
int main()
{
    int n;
    int capacity;
    int i;

    printf("Enter the number of items : ");
    scanf("%d",&n);

    int values[MAX_IEMS],weights[MAX_IEMS];

    printf("Enter the values and wights of the items:\n");
    for(i=0;i<n;i++)
    {
        printf("Enter value %d : ", i + 1);
        scanf("%d",&values[i]);

        printf("Enter weight %d : ", i+1);
        scanf("%d",&weights[i]);
    }

    printf("Enter the knapsack capacity :");
    scanf("%d",&capacity);

    int max_value = knapsack(values,weights,n,capacity);

    printf("Maximum value : %d \n",max_value);

    return 0;

}
```

INDO ASIAN ACADEMY

```
Enter the number of items : 4
Enter the values and wights of the items:
Enter value 1 : 8
Enter weight 1 : 2
Enter value 2 : 6
Enter weight 2 : 1
Enter value 3 : 16
Enter weight 3 : 3
Enter value 4 : 11
Enter weight 4 : 2
Enter the knapsack capacity :5
Selected Items:Item 4  Item 3
Maximum value : 27
```

INDO ASIAN ACADEMY

Lab 4 : Write a program to perform Knapsack problem using Greedy Solution.

```
#include <stdio.h>
#include <stdlib.h>

//function to solve knapsack problem using greedy technique

void kpgreedy(int M,int n, int w[], int P[])
{
    double x[n]; //solution vector 'x'
    double sum = 0.0; //total profit
    int i,j;

    //calculate profit-to-weight ratio for each object and store in
a seperate array
    double ratio[n];
    for(i=0;i<n;i++)
    {
        ratio[i]=(double)P[i]/w[i];
    }

    //create an array to store the order of objects
    int order[n];
    for(i=0;i<n;i++)
    {
        order[i]=i;
    }

    //sort the order of objects in descending order of p1/w1 ratio
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(ratio[order[j]]<ratio[order[j+1]])
            {
                //swap the order indices
                int tempIndex = order[j];
                order[j]=order[j+1];
                order[j+1]=tempIndex;
            }
        }
    }

    int Rc=M;    // Remaining capacity

    //Inititalize solution vector
    for(i=0;i<n;i++)
```

INDO ASIAN ACADEMY

```
    {
        x[i]=0;
    }

    //select order based on the described strategy

    for(i=0;i<n;i++)
    {
        int currentIndex = order[i];

        if(w[currentIndex]<=Rc)
        {
            x[currentIndex]=1;
            Rc = Rc - w[currentIndex];
            sum=sum+P[currentIndex];

        }else
        {
            x[currentIndex]=(double)Rc / w[currentIndex];
            sum+=(P[currentIndex] * x[currentIndex]);
            break;// All remaining objects cannot fit
        }
    }


    //Display the solution vector and the maximum profit

    printf("Solution vector 'x' : ");
    for(i=0;i<n;i++)
    {
        printf("%.2f",x[i]);

    }
    printf("\n Maximum Profit : %.21f \n",sum);
}

int main()
{
    int M,n,i;

    //Input knapsack capacity 'M' and the number of objects 'n'
    printf("Enter knapsack Capacity (M) :");
    scanf("%d",&M);
    printf("Enter number of Objects (n) :");
    scanf("%d",&n);

    int w[n],P[n];
```

INDO ASIAN ACADEMY

```
    //input weights and profits for each object
    printf("Enter weights for each object : \n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&w[i]);
    }

    printf("Enter profits for each object:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&P[i]);
    }

    //call the knapsack Greedy function to find the solution
    kpgreedy(M,n,w,P);

    return 0;
}
```

Output :

```
Enter knapsack Capacity (M) :40
Enter number of Objects (n) :4
Enter weights for each object :
20 25 10 15
Enter profits for each object:
20 40 35 45
Solution vector 'x' : 0.000.601.001.00
 Maximum Profit : 104.00000000000000000000
```

INDO ASIAN ACADEMY

Lab 5 : Write a program to implement the DFS and BFS algorithm for a graph.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int c[MAX][MAX];
int visited[MAX];
int queue[MAX];
int n; // number of vertices

void BFS(int v)
{
    int front=0, rear=-1, i;

    visited[v]=1;

    queue[++rear]=v;

    while(front<=rear)
    {
        v=queue[front++];
        printf("%d",v);

        for(i=1;i<=n;i++)
        {
            if(c[v][i]==1 && visited[i]==0)
            {
                queue[++rear]=i;
                visited[i]=1;
            }
        }
    }

   printf("\n");
}

void DFS(int v)
{
    int i;

    visited[v]=1;
    printf("%d",v);

    for(i=1; i<=n; i++)
    {
```

INDO ASIAN ACADEMY

```
        if(c[v][i]==1 && visited[i]==0)
        {
            DFS(i);
        }
    }
}


int main()
{
    int i,j,v;
    printf("Enter the number of vertices in the graph :");
    scanf("%d",&n);

    printf("Enter the cost of the matrix of the Graph :\n");
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
            scanf("%d",&c[i][j]);
    }

    for(i=1;i<=n;i++)
        visited[i]=0;

    printf("Enter the starting vertex for BFS :");
    scanf("%d",&v);

    printf("BFS traversal of the graph : ");
    BFS(v);

    //reset visited array for DFS
    for(i=1;i<=n;i++)
        visited[i]=0;

    printf("Enter the starting vertex for DFS :");
    scanf("%d",&v);

    printf("BFS traversal of the graph : ");
    DFS(v);

    printf("\n");

    return 0;
}
```

INDOASIAN ACADEMY

Output:

```
Enter the number of vertices in the graph :6
Enter the cost of the matrix of the Graph :
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 0 1
0 1 0 0 0 0
0 1 0 0 0 0
0 0 1 0 0 0
Enter the starting vertex for BFS :1
BFS traversal of the graph : 123456
Enter the starting vertex for DFS :1
BFS traversal of the graph : 124536
```

INDO ASIAN ACADEMY

Lab 6 : Write a program to find minimum and maximum value in an array using divide and conquer

```
#include <stdio.h>

void findMinMax(int arr[], int low, int high, int *min, int *max) {
    // Base case: If the array has only one element
    if (low == high) {
        *min = *max = arr[low];
        return;
    }

    // If the array has two elements, compare them
    if (high == low + 1) {
        if (arr[low] < arr[high]) {
            *min = arr[low];
            *max = arr[high];
        } else {
            *min = arr[high];
            *max = arr[low];
        }
        return;
    }

    // If the array has more than two elements, divide it and
recursively find min/max
    int midIndex = (low + high) / 2;
    int leftMin, leftMax, rightMin, rightMax;

    findMinMax(arr, low, midIndex, &leftMin, &leftMax);
    findMinMax(arr, midIndex + 1, high, &rightMin, &rightMax);

    // Merge the results
    *min = (leftMin < rightMin) ? leftMin : rightMin;
    *max = (leftMax > rightMax) ? leftMax : rightMax;
}

int main() {
    int n,i;

    //input size of the array
    printf("Enter the number of elements :");
    scanf("%d",&n);

    int arr[n];

    //input array elements
    printf("Enter the array elements :");
```

INDO ASIAN ACADEMY

```
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);

    int min, max;
    findMinMax(arr, 0, n - 1, &min, &max);

    printf("Minimum element: %d\n", min);
    printf("Maximum element: %d\n", max);

    return 0;
}
```

```
Enter the number of elements :5
Enter the array elements :10 30 20 50 40
Minimum element: 10
Maximum element: 50
```

INDO ASIAN ACADEMY

Lab 7: Write a test program to implement Divide and Conquer Strategy. Eg: Quicksort using divide and conquer strategy.

```
#include <stdio.h>


void quicksort(int A[], int low, int high)
{
    int j;
    if(low<high)
    {
        j=partition(A,low,high);
        quicksort(A,low,j-1);
        quicksort(A,j+1,high);
    }
}

int partition(int A[], int low, int high)
{
    int pivot, j, temp, i;
    pivot=low;
    i=low;
    j=high;

    while(i<j)
    {
        while(i<high && A[i]<=A[pivot])
            i++;

        while(A[j]>A[pivot])
            j--;

        if(i<j)
        {
            temp=A[i];
            A[i]=A[j];
            A[j]=temp;
        }
    }

    temp=A[pivot];
    A[pivot]=A[j];
    A[j]=temp;

    return j;
}
```

INDO ASIAN ACADEMY

```
int main()
{
    int i,n;
    printf("Enter the number of elements of the Array : ");
    scanf("%d",&n);

    int A[n];
    printf("Enter the elements of the Array : ");
    for(i=0;i<n;i++)
        scanf("%d",&A[i]);

    quicksort(A,0,n-1);

    printf("Sorted list of elements");
    for(i=0;i<n;i++)
        printf(" %d ",A[i]);

    return 0;
}
```

Output:

```
Enter the number of elements of the Array : 5
Enter the elements of the Array : 50 40 20 30 10
Sorted list of elements 10  20  30  40  50
```

INDO ASIAN ACADEMY

Lab 8 : Write a program to implement MergeSort algorithm for sorting a list of integers in ascending order.

```
#include <stdio.h>

void Merge(int A[], int low, int mid, int high)
{
    int i,j,k, c[high+1];
    i = low;
    k=low;
    j=mid+1;

    while((i<=mid)&&(j<=high))
    {
        if(A[i] <= A[j])
        {
            c[k]=A[i];
            i++;
            k++;
        }else
        {
            c[k]=A[j];
            j++;
            k++;

        }

     }
     while(i<=mid)
     {
         c[k]=A[i];
            k++;
            i++;

     }
     while(j<=high)
     {
         c[k]=A[i];
            k++;
            j++;

     }

     for(i=low; i<=k-1; i++ )
     {
         A[i] = c[i];
     }
```

INDO ASIAN ACADEMY

```
}

void MergeSort(int A[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high)/2;
        MergeSort(A,low, mid);
        MergeSort(A, mid+1, high);
        Merge(A,low,mid,high);
    }
}

int main()
{
    int i, n;

    printf("Enter the elements of the Array : ");
    scanf("%d", &n);

    int A[n];

    printf("Enter the elements of the Array :");
    for(i=0;i<n;i++)
        scanf("%d",&A[i]);

    MergeSort(A,0,n-1);

    printf(" Sorted elements of the Array :");
    for(i=0;i<n;i++)
        printf(" %d ",A[i]);

    printf("\n");

    return 0;


}
```

Output :

```
Enter the elements of the Array : 5
Enter the elements of the Array :50 30 20 40 10
 Sorted elements of the Array : 10  20  30  40  50
```

INDO ASIAN ACADEMY

Lab 9: Sort a given set of n integers using Merge Sort method and compute its time complexity. Run the program for varied values of n>5000, and record the time taken to sort.

```
#include <stdio.h>
#include <time.h>


void Merge(int A[], int low, int mid, int high)
{
    int i,j,k, c[high+1];
    i = low;
    k=low;
    j=mid+1;

    while((i<=mid)&&(j<=high))
    {
        if(A[i] <= A[j])
        {
            c[k]=A[i];
            i++;
            k++;
        }else
        {
            c[k]=A[j];
            j++;
            k++;

        }

     }
     while(i<=mid)
     {
         c[k]=A[i];
            k++;
            i++;

     }
     while(j<=high)
     {
         c[k]=A[i];
            k++;
            j++;

     }

     for(i=low; i<=k-1; i++ )
     {
```

INDO ASIAN ACADEMY

```
          A[i] = c[i];
      }

}

void MergeSort(int A[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high)/2;
        MergeSort(A,low, mid);
        MergeSort(A, mid+1, high);
        Merge(A,low,mid,high);
    }
}

int main()
{
    int n,i;
    printf("Enter the number of elements (n) or -999 to quit :");
    scanf("%d",&n);

    while(n!=-999)
    {
        int A[n];

        //seed the random number generator
        srand(time(NULL));

        //Generate random numbers for the array
        for(i=0;i<n;i++)
        {
            A[i]=rand();

        }

        //record the start time
        clock_t start_time = clock();

        //perform Merge Sort
        MergeSort(A,0,n-1);

        //record the end time
        clock_t end_time = clock();

        //Calculate time taken
```

INDO ASIAN ACADEMY

```
        double time_taken = (double)(end_time - start_time) /
CLOCKS_PER_SEC;

        printf("Time taken to sort ( n = %d ) : %lf seconds
\n",n,time_taken);

        printf("Enter the number of elements (n) or -999 to quit
:");
        scanf("%d",&n);
    }
    return 0;
}
```

Output :

```
Enter the number of elements (n) or -999 to quit :100
Time taken to sort ( n = 100 ) : 0.000000 seconds
Enter the number of elements (n) or -999 to quit :1000
Time taken to sort ( n = 1000 ) : 0.000000 seconds
Enter the number of elements (n) or -999 to quit :5000
Time taken to sort ( n = 5000 ) : 0.001000 seconds
Enter the number of elements (n) or -999 to quit :10000
Time taken to sort ( n = 10000 ) : 0.002000 seconds
Enter the number of elements (n) or -999 to quit :50000
Time taken to sort ( n = 50000 ) : 0.014000 seconds
Enter the number of elements (n) or -999 to quit :75000
Time taken to sort ( n = 75000 ) : 0.028000 seconds
Enter the number of elements (n) or -999 to quit :
```

INDO ASIAN ACADEMY

Lab 10: Sort a given set of n integers using Quick Sort method and compute its time complexity. Run the program for varied values of n>5000, and record the time taken to sort.

```
#include <stdio.h>
#include <time.h>

void quicksort(int A[], int low, int high)
{
    int j;
    if(low<high)
    {
        j=partition(A,low,high);
        quicksort(A,low,j-1);
        quicksort(A,j+1,high);
    }
}

int partition(int A[], int low, int high)
{
    int pivot, j, temp, i;
    pivot=low;
    i=low;
    j=high;

    while(i<j)
    {
        while(i<high && A[i]<=A[pivot])
            i++;

        while(A[j]>A[pivot])
            j--;

        if(i<j)
        {
            temp=A[i];
            A[i]=A[j];
            A[j]=temp;
        }
    }

    temp=A[pivot];
    A[pivot]=A[j];
    A[j]=temp;

    return j;
}
```

INDO ASIAN ACADEMY

```
int main()
{
    int n,i;
    printf("Enter the number of elements (n) or -999 to quit :");
    scanf("%d",&n);

    while(n!=-999)
    {
        int A[n];

        //seed the random number generator
        srand(time(NULL));

        //Generate random numbers for the array
        for(i=0;i<n;i++)
        {
            A[i]=rand();

        }

        //record the start time
        clock_t start_time = clock();

        //perform Merge Sort
        quicksort(A,0,n-1);

        //record the end time
        clock_t end_time = clock();

        //Calculate time taken
        double time_taken = (double)(end_time - start_time) /
CLOCKS_PER_SEC;

        printf("Time taken to sort ( n = %d ) : %lf seconds
\n",n,time_taken);

        printf("Enter the number of elements (n) or -999 to quit
:");
        scanf("%d",&n);
    }
    return 0;
}
```

INDO ASIAN ACADEMY

Output:

```
Enter the number of elements (n) or -999 to quit :5000
Time taken to sort ( n = 5000 ) : 0.001000 seconds
Enter the number of elements (n) or -999 to quit :10000
Time taken to sort ( n = 10000 ) : 0.002000 seconds
Enter the number of elements (n) or -999 to quit :50000
Time taken to sort ( n = 50000 ) : 0.007000 seconds
Enter the number of elements (n) or -999 to quit :75000
Time taken to sort ( n = 75000 ) : 0.010000 seconds
Enter the number of elements (n) or -999 to quit :
```

******Part A Ends ****

INDO ASIAN ACADEMY

Part – B

Lab 11: Write a C Program that accepts the vertices and edges for a graph and stores it as an adjacency matrix.

Code :

```
#include <stdio.h>

int main()
{
    int adj[10][10]; // adjacency matrix
    int n;           // number of vertices
    int max_edges, origin, destin, i, j;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    max_edges = n * (n - 1) / 2;

    // Initialize the adjacency matrix to 0
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
       {
            adj[i][j] = 0;
        }
    }

    // Input edges
    for (i = 1; i <= max_edges; i++)
    {
        printf("\nEnter edge [ %d ] ( -1 -1 to quit ): ", i);
        scanf("%d %d", &origin, &destin);

        if ((origin == -1) && (destin == -1))
            break;

        if (origin >= n || destin >= n || origin < 0 || destin < 0)
        {
            printf("\nInvalid vertex!\n");
            i--; // decrement i to repeat this iteration
        } else
          {
            // Set both adj[origin][destin] and adj[destin][origin] to 1
            adj[origin][destin] = 1;
            adj[destin][origin] = 1;
        }
    }

    // Print the adjacency matrix
    printf("\nThe adjacency matrix is:\n");
```

INDO ASIAN ACADEMY

```
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%4d", adj[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter the number of vertices: 5

Enter edge [ 1 ] ( -1 -1 to quit ): 0 1

Enter edge [ 2 ] ( -1 -1 to quit ): 0 2

Enter edge [ 3 ] ( -1 -1 to quit ): 0 3

Enter edge [ 4 ] ( -1 -1 to quit ): 1 3

Enter edge [ 5 ] ( -1 -1 to quit ): 2 3

Enter edge [ 6 ] ( -1 -1 to quit ): 3 4

Enter edge [ 7 ] ( -1 -1 to quit ): 4 2

Enter edge [ 8 ] ( -1 -1 to quit ): -1 -1

The adjacency matrix is:
   0   1   1   1   0
   1   0   0   1   0
   1   0   0   1   1
   1   1   1   0   1
   0   0   1   1   0
```

INDO ASIAN ACADEMY

Lab 12 - Write program to implement greedy algorithm for job sequencing with deadlines.

Code :

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

// A structure to represent a job
typedef struct Job
       {
      char id; // Job Id
      int dead; // Deadline of job
      int profit; // Profit if job is over before or on
                        // deadline
} Job;

// This function is used for sorting all jobs according to  profit
int compare(const void* a, const void* b)
      {
      Job* temp1 = (Job*)a;
      Job* temp2 = (Job*)b;
      return (temp2->profit - temp1->profit);
      }

// Find minimum between two numbers.
int min(int num1, int num2)
     {
      return (num1 > num2) ? num2 : num1;
     }

// Returns maximum profit from jobs
void printJobScheduling(Job arr[], int n)
{
      // Sort all jobs according to decreasing order of profit
      qsort(arr, n, sizeof(Job), compare);

      int result[n]; // To store result (Sequence of jobs)
      bool slot[n]; // To keep track of free time slots

      // Initialize all slots to be free
      for (int i = 0; i < n; i++)
            slot[i] = false;

      // Iterate through all given jobs
      for (int i = 0; i < n; i++)
                  {
```

INDO ASIAN ACADEMY

```
            // Find a free slot for this job (Note that we start  from the
last possible slot)
            for (int j = min(n, arr[i].dead) - 1; j >= 0; j--)
                                    {
                  // Free slot found
                  if (slot[j] == false)
                                          {
                        result[j] = i; // Add this job to result
                        slot[j] = true; // Make this slot occupied
                        break;
                  }
            }
      }

      // Print the result
      for (int i = 0; i < n; i++)
            if (slot[i])
                  printf("%c ", arr[result[i]].id);
}

// Driver's code
int main()
{
      Job arr[] = { { 'a', 2, 100 },
                    { 'b', 1, 19 },
                    { 'c', 2, 27 },
                    { 'd', 1, 25 },
                    { 'e', 3, 15 } };
      int n = sizeof(arr) / sizeof(arr[0]);
      printf(    "Following is maximum profit sequence of jobs \n");

      // Function call
      printJobScheduling(arr, n);
      return 0;
}
```

OUTPUT

```
Following is maximum profit sequence of jobs
c a e
Process returned 0 (0x0)   execution time : 4.267 s
Press any key to continue.
```

INDO ASIAN ACADEMY

Lab 13 - Write program to implement Dynamic Programming algorithm for the Optimal Binary Search Tree Problem.

Code :

```
#include<stdio.h>
#define INT_MAX  500
int sum(int freq[], int low, int high)
 {    //sum of frequency from low to high range
   int sum = 0;
   for (int k = low; k <=high; k++)
      sum += freq[k];
   return sum;
}

int minCostBST(int keys[], int freq[], int n)
 {
   int cost[n][n];

   for (int i = 0; i < n; i++)
      cost[i][i] = freq[i];

   for (int length=2; length<=n; length++)

   {
      for (int i=0; i<=n-length+1; i++)
           {    //from 0th row to n-length+1 row as i
               int j = i+length-1;
              cost[i][j] = INT_MAX;    //initially store to maximum value

         for (int r=i; r<=j; r++)
        {
            //find cost when r is root of subtree
            int c = ((r > i)?cost[i][r-1]:0)+((r <
j)?cost[r+1][j]:0)+sum(freq, i, j);
            if (c < cost[i][j])
               cost[i][j] = c;
            }
      }
   }
   return cost[0][n-1];
}

int main() {
   int keys[] = {10, 12, 20};
   int freq[] = {34, 8, 50};
   int n = 3;
   printf("Cost of Optimal BST is:  %d ", minCostBST(keys, freq, n));
}
```

INDO ASIAN ACADEMY

OUTPUT

```
Cost of Optimal BST is:  142
Process returned 0 (0x0)   execution time : 2.420 s
Press any key to continue.
```

INDO ASIAN ACADEMY

Lab 14 - Write a program that implements Prim's algorithm to generate minimum cost spanning Tree.

ALGORITHM

A spanning tree is a sub-graph of an undirected connected graph, which includes all the vertices of the graph with a minimum possible number of edges.

A minimum spanning tree (MST) is a subset of the edges of a connected, edge-weighted graph that connects all the vertices together without any cycles and with the minimum possible total edge weight

The steps for implementing Prim's algorithm are as follows:

Step 1: Initialize the minimum spanning tree with a vertex chosen at random.
Step 2: Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree

Step 3: Keep repeating step 2 until we get a minimum spanning tree

CODE

```
#include <stdio.h>
#include <limits.h>
#define vertices 5  /*Define the number of vertices in the graph*/
/* create minimum_key() method for finding the vertex that has minimum key-value
and that is not added in MST yet */
int minimum_key(int k[], int mst[])
{
    int minimum  = INT_MAX, min,i;

    /*iterate over all vertices to find the vertex with minimum key-value*/
    for (i = 0; i < vertices; i++)
        if (mst[i] == 0 && k[i] < minimum )
            minimum = k[i], min = i;
    return min;
}
/* create prim() method for constructing and printing the MST.
The g[vertices][vertices] is an adjacency matrix that defines the graph for MST.*/
void prim(int g[vertices][vertices])
{
    /* create array of size equal to total number of vertices for storing the
MST*/
    int parent[vertices];
    /* create k[vertices] array for selecting an edge having minimum weight*/
    int k[vertices];
    int mst[vertices];
    int i, count,edge,v; /*Here 'v' is the vertex*/
    for (i = 0; i < vertices; i++)
    {
        k[i] = INT_MAX;
```

INDOASIAN ACADEMY

```
        mst[i] = 0;
    }
    k[0] = 0; /*It select as first vertex*/
    parent[0] = -1;   /* set first value of parent[] array to -1 to make it root
of MST*/
    for (count = 0; count < vertices-1; count++)
    {
        /*select the vertex having minimum key and that is not added in the MST
yet from the set of vertices*/
        edge = minimum_key(k, mst);
        mst[edge] = 1;
        for (v = 0; v < vertices; v++)
        {
            if (g[edge][v] && mst[v] == 0 && g[edge][v] <  k[v])
            {
                parent[v]  = edge, k[v] = g[edge][v];
            }
        }
     }
     /*Print the constructed Minimum spanning tree*/
     printf("\n Edge \t  Weight\n");
     for (i = 1; i < vertices; i++)
     printf(" %d <-> %d    %d \n", parent[i], i, g[i][parent[i]]);

}
int main()
{
    int g[vertices][vertices] = {{0, 0, 3, 0, 0},
                                 {0, 0, 10, 4, 0},
                                 {3, 10, 0, 2, 6},
                                 {0, 4, 2, 0, 1},
                                 {0, 0, 6, 1, 0},
                                 };

    prim(g);
    return 0;
}
```

OUTPUT

```
 Edge     Weight
 3 <-> 1    4
 0 <-> 2    3
 2 <-> 3    2
 3 <-> 4    1

Process returned 0 (0x0)   execution time : 1.400 s
Press any key to continue.
```

DO ASIAN ACADEMY

Lab 15 - Write a program that implements Kruskal's algorithm to generate minimum cost spanning tree.

ALGORITHM

A spanning tree is a sub-graph of an undirected connected graph, which includes all the vertices of the graph with a minimum possible number of edges.

A minimum spanning tree (MST) is a subset of the edges of a connected, edge-weighted graph that connects all the vertices together without any cycles and with the minimum possible total edge weight.

Step 1: Sort all the edges in non-decreasing order of their weight.

Step 2: Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.

Step 3 : Repeat step#2 until there are (V-1) edges in the spanning tree

PROGRAM

```
/* Program to implement Krushkal Algorithm*/
#include <stdio.h>

#define MAX 30

typedef struct edge
{
  int u, v, w;
} edge;

typedef struct edge_list
{
  edge data[MAX];
  int n;
} edge_list;

edge_list elist;

int Graph[MAX][MAX], n;
edge_list spanlist;

void kruskalAlgo();
int find(int belongs[], int vertexno);
void applyUnion(int belongs[], int c1, int c2);
void sort();
void print();
```

INDO ASIAN ACADEMY

```
// Applying Krushkal Algo
void kruskalAlgo()
{
  int belongs[MAX], i, j, cno1, cno2;
  elist.n = 0;

  for (i = 1; i < n; i++)
    for (j = 0; j < i; j++)

       {
      if (Graph[i][j] != 0)
         {
        elist.data[elist.n].u = i;
        elist.data[elist.n].v = j;
        elist.data[elist.n].w = Graph[i][j];
        elist.n++;
      }
    }

  sort();

  for (i = 0; i < n; i++)
    belongs[i] = i;

  spanlist.n = 0;

  for (i = 0; i < elist.n; i++)
  {
    cno1 = find(belongs, elist.data[i].u);
    cno2 = find(belongs, elist.data[i].v);

    if (cno1 != cno2)
       {
      spanlist.data[spanlist.n] = elist.data[i];
      spanlist.n = spanlist.n + 1;
      applyUnion(belongs, cno1, cno2);
    }
  }
}

int find(int belongs[], int vertexno)
 {
  return (belongs[vertexno]);
}

void applyUnion(int belongs[], int c1, int c2)
{
  int i;

  for (i = 0; i < n; i++)
    if (belongs[i] == c2)
```

INDO ASIAN ACADEMY

```
      belongs[i] = c1;
}

// Sorting algo
void sort()
 {
  int i, j;
  edge temp;

  for (i = 1; i < elist.n; i++)
    for (j = 0; j < elist.n - 1; j++)
      if (elist.data[j].w > elist.data[j + 1].w)
         {
        temp = elist.data[j];
        elist.data[j] = elist.data[j + 1];
        elist.data[j + 1] = temp;
      }
}

// Printing the result
void print()
 {
  int i, cost = 0;

  for (i = 0; i < spanlist.n; i++)
  {
    printf("\n%d - %d : %d", spanlist.data[i].u, spanlist.data[i].v,
spanlist.data[i].w);
    cost = cost + spanlist.data[i].w;
  }

  printf("\nSpanning tree cost: %d", cost);
}

int main()
{
  int i, j, total_cost;

  n = 6;

  Graph[0][0] = 0;
  Graph[0][1] = 4;
  Graph[0][2] = 4;
  Graph[0][3] = 0;
  Graph[0][4] = 0;
  Graph[0][5] = 0;
  Graph[0][6] = 0;

  Graph[1][0] = 4;
  Graph[1][1] = 0;
  Graph[1][2] = 2;
```

INDO ASIAN ACADEMY

```
  Graph[1][3] = 0;
  Graph[1][4] = 0;
  Graph[1][5] = 0;
  Graph[1][6] = 0;

  Graph[2][0] = 4;
  Graph[2][1] = 2;
  Graph[2][2] = 0;
  Graph[2][3] = 3;
  Graph[2][4] = 4;
  Graph[2][5] = 0;
  Graph[2][6] = 0;

  Graph[3][0] = 0;
  Graph[3][1] = 0;
  Graph[3][2] = 3;
  Graph[3][3] = 0;
  Graph[3][4] = 3;
  Graph[3][5] = 0;
  Graph[3][6] = 0;

  Graph[4][0] = 0;
  Graph[4][1] = 0;
  Graph[4][2] = 4;
  Graph[4][3] = 3;
  Graph[4][4] = 0;
  Graph[4][5] = 0;
  Graph[4][6] = 0;

  Graph[5][0] = 0;
  Graph[5][1] = 0;
  Graph[5][2] = 2;
  Graph[5][3] = 0;
  Graph[5][4] = 3;
  Graph[5][5] = 0;
  Graph[5][6] = 0;

  kruskalAlgo();
  print();
}
```

OUTPUT

```
2 - 1 : 2
5 - 2 : 2
3 - 2 : 3
4 - 3 : 3
1 - 0 : 4
Spanning tree cost: 14
Process returned 0 (0x0)   execution time : 2.213 s
Press any key to continue.
```

INDO ASIAN ACADEMY

Lab 16 - Write program to implement backtracking algorithm for solving problems like N queens.

ALGORITHM

The **N** Queen is the problem of placing **N** chess queens on an **N×N** chessboard so that no two queens attack each other.

For example, the following is a solution for the 4 Queen problem.

The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.

Step 1: Start in the leftmost column. If all queens are placed return true
Step 2: Try all rows in the current column. Do the following steps for every row.

Step 3: If the queen can be placed safely in this row
Then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.

Step 4: If placing the queen in [row, column] leads to a solution then return true.

Step 5: If placing queen doesn't lead to a solution then unmark this [row, column] then backtrack and try other rows.

If all rows have been tried and valid solution is not found return false to trigger backtracking.

CODE

```
/* program to solve N Queen Problem using backtracking */
#define N 4
#include <stdbool.h>
#include <stdio.h>

void printSolution(int board[N][N])
{
     for (int i = 0; i < N; i++) {
          for (int j = 0; j < N; j++)
               printf(" %d ", board[i][j]);
          printf("\n");
```

INDO ASIAN ACADEMY

```
      }
}

/* A utility function to check if a queen can  be placed on
board[row][col]. Note that this function is called when "col" queens are
already placed in columns from 0 to col -1. So we need to check only left
side for attacking queens */

bool isSafe(int board[N][N], int row, int col)
{
      int i, j;

      /* Check this row on left side */
      for (i = 0; i < col; i++)
            if (board[row][i])
                  return false;

      /* Check upper diagonal on left side */
      for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
            if (board[i][j])
                  return false;

      /* Check lower diagonal on left side */
      for (i = row, j = col; j >= 0 && i < N; i++, j--)
            if (board[i][j])
                  return false;

      return true;
}

/* A recursive utility function to solve N Queen problem */
bool solveNQUtil(int board[N][N], int col)
{
      /* base case: If all queens are placed  then return true */
      if (col >= N)
            return true;

      /* Consider this column and try placing  this queen in all rows one
by one */
      for (int i = 0; i < N; i++)
                              {
            /* Check if the queen can be placed on board[i][col] */
            if (isSafe(board, i, col))
                   {
                  /* Place this queen in board[i][col] */
                  board[i][col] = 1;

                  /* recur to place rest of the queens */
                  if (solveNQUtil(board, col + 1))
                        return true;
```

INDO ASIAN ACADEMY

```
                /* If placing queen in board[i][col]  doesn't lead to a
solution, then
                remove queen from board[i][col] */
                board[i][col] = 0; // BACKTRACK
            }
        }

        /* If the queen cannot be placed in any row in
            this column col then return false */
        return false;
}


/* This function solves the N Queen problem using Backtracking. It mainly
uses solveNQUtil() to
solve the problem. It returns false if queens cannot be placed, otherwise,
return true and
prints placement of queens in the form of 1s. Note that there may be more
than one
solutions, this function prints one of the
feasible solutions.*/

bool solveNQ()
{
        int board[N][N] = { { 0, 0, 0, 0 },
                            { 0, 0, 0, 0 },
                            { 0, 0, 0, 0 },
                            { 0, 0, 0, 0 } };

        if (solveNQUtil(board, 0) == false)
            {
            printf("Solution does not exist");
            return false;
        }

        printSolution(board);
        return true;
}
int main()
{
        solveNQ();
        return 0;
}
```

OUTPUT

```
0  0  1  0
1  0  0  0
0  0  0  1
0  1  0  0
```

****

INDO ASIAN ACADEMY